

MLIR-AIR: From Loop Nests to Silicon

Detailed Notes on Mapping AI Workloads onto AMD NPUs

Adithya Jillellamudi

September 2026

Contents

1	What is MLIR-AIR?	3
1.1	Frontend Stays the Same	3
2	Why Spatial Hardware Needs a New IR	3
2.1	Standard IR vs. AIR: The Core Difference	3
2.2	Why Standard IR Can't Target Spatial Hardware	4
3	Hardware Trends Motivating AIR	4
3.1	Multi-root Memory Hierarchy (NUMA at Chip Scale)	4
3.2	DMA: Mechanism vs. Policy	5
3.3	Asynchronous Execution	5
4	Four Compiler Requirements for Spatial Hardware	5
5	Comparison with Related Frameworks	6
5.1	IRON: Close-to-Metal Toolkit	6
5.2	ARIES: Agile Compilation Flow	6
6	ACDG: Asynchronous Control and Dataflow Graph	6
6.1	What It Is	6
6.2	Token Mechanism	6
6.3	Dependency Types (RAW, WAR, WAW)	7
6.4	Why CPU Hazard Resolution Fails on NPUs	7
6.5	Three Kinds of Synchronization Edges	8
6.5.1	Concrete Example: Double-Buffered Matmul Tile	8
6.5.2	Why Three Edges, Not Just Dependency?	9
6.6	Composing with Loops: Software Pipelining	9
7	AIR Dialect Constructs in Use: Vector-Add Walkthrough	10
7.1	Stage 0: The Algorithm (What We Actually Want to Compute)	10
7.2	Stage 1: Generic MLIR Input (Listing 7, p.33)	10
7.2.1	Step 1: Function Signature — Three Flat Buffers	10
7.2.2	Step 2: Parallelization — Split Work, But No Placement	11
7.2.3	Step 3: Tiling — Local Buffers, But Opaque Memory	11
7.2.4	Step 4: Data Movement — Coupled, Single-Sided	11
7.2.5	Step 5: Compute — Correct, But Sequential	12
7.2.6	Step 6: Write-Back and Cleanup	12
7.2.7	Summary of What Generic MLIR Captures vs. Misses	12

7.3	Stage 2: AIR-Transformed Output (Listing 3, p.14)	12
7.3.1	Step 1: Channel Declarations (Replacing <code>memref.copy</code>)	12
7.3.2	Step 2: Launch + Segment (Replacing <code>scf.parallel</code> Outer Scope)	13
7.3.3	Step 3: Segment-Level DMAs ($L3 \leftrightarrow L2$)	13
7.3.4	Step 4: Herd Launch (Replacing <code>scf.parallel</code> Body)	14
7.3.5	Step 5: Compute Loop with Pipelining (Replacing Sequential <code>scf.for</code>)	14
7.3.6	Step 6: Channel Gets Inside Herd ($L2 \rightarrow L1$)	14
7.3.7	Step 7: Async Compute (Same Math, Explicit Dependencies)	15
7.3.8	Step 8: Write-Back and Final Sync	15
7.4	Stage 3: What the Compiler Added	15
8	Scheduling Constructs	16
8.1	The Problem: Where Does Computation Run?	16
8.2	<code>air.launch</code> — Host-to-Accelerator Boundary	16
8.3	<code>air.segment</code> — Resource Reservation	16
8.4	<code>air.herd</code> — Parallel Workers on Physical Cores	17
8.5	Hierarchy Maps to Memory Hierarchy	17
9	Data Locality Constructs	17
9.1	The Problem: No Caches, No Automatic Data Movement	17
9.2	Memory Hierarchy via MemRef Annotations	18
9.3	<code>air.memcpy</code> — Transitional Abstraction	18
9.4	<code>air.channel</code> — Core Data Movement Primitive	18
9.5	How Scheduling and Data Locality Interact	19
10	Synchronization Constructs (Complete)	19
10.1	<code>air.token</code> — Explicit Dependency Control	20
10.1.1	Mechanism 1: <code>air.wait_all</code> — Blocking Barrier	20
10.1.2	Mechanism 2: Synchronization Lists — Scheduling Graph	20
10.2	Three Types of Synchronization Edges	20
10.2.1	Dependency Lists — Directed, “Happens-Before”	20
10.2.2	Concurrency Lists — Undirected, “Must Co-Schedule”	20
10.2.3	Affinity Lists — Undirected, “Same Resources, Different Time”	21
10.3	Tokens in Loops	21
10.3.1	<code>scf.for</code> — Single Token (Fig 4a)	21
10.3.2	<code>scf.for</code> — Multiple Tokens (Fig 4b)	21
10.3.3	<code>scf.parallel</code> — Fork/Join (Fig 4c)	21
10.4	<code>air.channel</code> as Synchronization	22
10.5	Grouped Completion via <code>air.launch</code>	22
10.6	Hardware Pipelining: Ping-Pong Buffering	22
10.7	Synchronization Summary	23
11	Summary: The AIR Mental Model	23

1 What is MLIR-AIR?

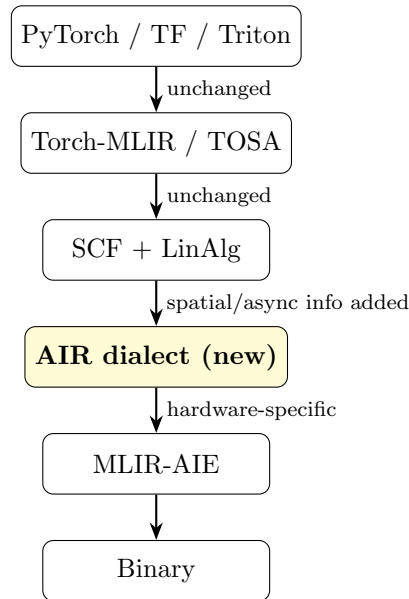
MLIR-AIR is a new MLIR dialect that bridges the gap between high-level AI workloads (written in PyTorch, TensorFlow, Triton) and fine-grained spatial architectures like AMD’s NPUs. The key idea: traditional compiler IRs hide parallelism, data movement, and synchronization behind abstractions like shared memory and thread schedulers. Spatial hardware has none of that—no caches, no coherence, no central scheduler. AIR makes all three **explicit in the IR itself**.

Three things AIR exposes that traditional IRs hide:

1. **Spatial structure** — *where* computation runs (which tile, which core, which region of the chip). Expressed via `air.herd` and `air.segment`.
2. **Temporal structure** — *when* things happen. Expressed via `air.token` dependency edges, not implicit memory ordering.
3. **Data movement as first-class operations** — `air.channel.put/air.channel.get` are separate IR ops. Compiler can see, schedule, and overlap them with compute.

1.1 Frontend Stays the Same

AIR does *not* change the programming model. The pipeline is:



AIR is a **new middle layer** inserted between generic MLIR and hardware-specific MLIR-AIE. It doesn’t change anything above it. The compiler *infers* spatial structure from loop nests and tiling decisions, then *encodes* it as AIR operations.

2 Why Spatial Hardware Needs a New IR

2.1 Standard IR vs. AIR: The Core Difference

Standard IRs (LLVM IR, MLIR SCF+memref) model programs as:

- **Shared memory** — all threads see one big address space. Write to address X, any thread reads X. Hardware caches and coherence protocols make this work.
- **Centralized scheduling** — you say “run 1000 threads.” One scheduler (OS on CPU, warp scheduler on GPU) maps threads to cores. You don’t control placement.

In this model, the IR looks like:

```
%val = memref.load %buf[%i]      // which memory? don't care
%res = arith.muli %val, %val      // on which core? don't care
memref.store %res, %out[%i]      // when visible? hardware sorts it out
```

No information about where things physically are, who moves data, or what depends on what (beyond program order).

AIR flips this. Same operation in AIR:

```
%tok0 = air.channel.get %L1_buf from @chan_A // explicit DMA: L2 → L1
air.herd [%x, %y] in [4, 4] {                // explicit: 4x4 cores
    %tok1 = air.wait %tok0                    // explicit: wait for
        DMA
    linalg.matmul %L1_A, %L1_B, %L1_C         // compute on local data
    air.channel.put %L1_C to @chan_out        // explicit DMA: L1 → L2
}
```

Every question the standard IR leaves to hardware—*where* is the data, *who* moves it, *which* cores compute, *what* waits for *what*—AIR answers explicitly.

2.2 Why Standard IR Can't Target Spatial Hardware

If you give a spatial NPU code that says “load from memory,” the hardware doesn't know which memory, through which path, into which scratchpad. There's no cache. No coherence. No central scheduler. The IR *must* carry this information or the backend has nothing to work with.

3 Hardware Trends Motivating AIR

3.1 Multi-root Memory Hierarchy (NUMA at Chip Scale)

Traditional compilers treat main memory as one coherent root. Modern devices use multiple independent memory channels for aggregate bandwidth. These channels are *physically separate paths to different parts of memory*. No hardware coherence between them.

Example: Matrix multiply on a multi-chiplet NPU. Matrix B shared across cores. B lives behind chiplet 0's memory channel.

- Chiplet 0 reads B fast—local channel, short path
- Chiplet 1 reads same B **slow**—across interconnect, contends for chiplet 0's memory port

Even with co-design, NUMA effects creep in at scale because:

- **Wire delay scales with distance.** Core at top-left of a 10×10 grid is far from memory at bottom-right (~1cm/ns on silicon).
- **Bandwidth is finite.** One memory port can't feed 100 cores. Partition into banks ⇒ non-uniform access.
- **Multi-chiplet is often necessary.** Yield drops with die size, so chiplets connected by inter-chiplet links (slower than on-die wires).

3.2 DMA: Mechanism vs. Policy

DMA *moves* data but doesn't *decide* what to move, when, or where.

1. **DMA doesn't know about duplicates.** 8 cores needing tile B: naive compiler issues 8 DMAs from remote memory. Smart compiler: one DMA to local memory, 8 local reads.
2. **DMA doesn't schedule itself.** Fire too late $\Rightarrow$ cores stall. Too early $\Rightarrow$ local buffer overflow. Compiler must pipeline prefetches.
3. **DMA doesn't know topology.** Two DMAs sharing a path contend. DMA engine doesn't throttle across channels.

The compiler is the logistics planner; DMA is the truck. AIR models the topology so the compiler can make good placement/scheduling decisions.

3.3 Asynchronous Execution

Each NPU tile has multiple **independent actors**:

- **Compute engine** — vector/scalar ALU
- **DMA engine** — moves data between memory levels
- **NoC switch** — routes data between tiles

These operate concurrently. Naive sequential execution wastes half the time (one actor idle while another works). Async execution pipelines them:

	DMA get	Compute	DMA put
Iter 0	[=====]	[=====]	[=====]
Iter 1	[=====]	[=====]	[=====]
Iter 2	[=====]	[=====]	[=====]

Each actor has a local hardware scheduler with a task queue. When a dependency is satisfied (token fires), it picks next task. No central coordinator. `air.token` encodes these dependencies in the IR.

4 Four Compiler Requirements for Spatial Hardware

The paper identifies four capabilities a compiler framework must have (Section 2.2):

1. **Memory allocation control.** Allocate memory at specific levels (L1/L2/DRAM) *and* specific NUMA domains within each level. Not just `malloc(size)` but “put this in L1 of tile (2,3)” or “L2 of cluster 0.” In AIR: `memref` types with memory space annotations.
2. **Compute placement control.** Describe which cores form a group, run concurrently, share resources for cheap sync. In AIR: `air.herd` places compute on a core grid, `air.segment` groups herds sharing resources.
3. **Separate data movement from computation.** Make DMA ops visible in the IR so compiler can schedule them independently, overlap with compute, apply architecture-specific optimizations. In AIR: `air.channel.put/get` are separate IR nodes.
4. **Dependency resolution close to hardware.** Express dependencies explicitly so hardware resolvers (not software polling) can react in nanoseconds. In AIR: `air.token` values $\rightarrow$ lowered to hardware lock signals on tile task queues.

5 Comparison with Related Frameworks

5.1 IRON: Close-to-Metal Toolkit

IRON is a Python toolkit for AMD NPUs. You manually control everything: tile placement, DMA configuration, synchronization locks. Key abstraction: `ObjectFifo`—a circular buffer between producer and consumer with built-in lock synchronization.

	IRON	AIR
Abstraction level	Assembly-like	C-like
Placement	Manual	Compiler-managed
DMA config	Hand-written descriptors	Compiler-inferred
Target	Single kernels	Complex fused workloads
IR level	<code>mlir-aie</code> (bottom)	AIR dialect (middle)

Both target the same `mlir-aie` backend. AIR automates what IRON makes you do by hand. IRON is for peak performance on small kernels; AIR makes complex workloads (like full attention blocks, ~150 lines) feasible.

5.2 ARIES: Agile Compilation Flow

ARIES is an MLIR-based compilation flow for AIE architectures with a Triton-like programming model (tile-based task abstraction with scheduling primitives like `.to()`, `.pipeline()`, `.vectorize()`).

ARIES is modular: unified MLIR IR covering AIE core, AIE array, and PL (FPGA), with portable backends for both Versal ACAP and Ryzen NPU. It reuses existing MLIR dialects (SCF, memref, affine, AIEVec, ADF).

The paper’s specific distinction: ARIES’s spatial intelligence lives in the *frontend API* (Python scheduling primitives) and *lowering pipeline*. AIR encodes spatial/async semantics as *new IR operations* (`air.herd`, `air.channel`, `air.token`) visible to any MLIR pass. Whether this distinction matters practically is debatable—both achieve good GEMM results.

6 ACDG: Asynchronous Control and Dataflow Graph

6.1 What It Is

A DAG embedded directly in the MLIR-AIR IR:

- **Nodes** = MLIR operations (compute, DMA, sync)
- **Edges** = dependencies encoded via `air.token` SSA values

Not a separate data structure. Uses MLIR’s native SSA (Static Single Assignment) form: every `air.token` value is defined exactly once, and any operation consuming it must be dominated by its definition. MLIR’s existing verifier checks dependency correctness *for free*.

6.2 Token Mechanism

An operation marked **async produces** a token. An operation that **consumes** that token can’t execute until the producer finishes:

```
%tok0 = air.channel.get async @chan, %buf    // produces tok0
%tok1 = linalg.matmul async [%tok0] ...      // waits for tok0
%tok2 = air.channel.put async [%tok1] ...    // waits for tok1
```

Independent ops have no edge $\Rightarrow$ run in parallel:

```
%tok0 = air.channel.get async @chanA, %bufA // fetch A
%tok1 = air.channel.get async @chanB, %bufB // fetch B (no dep on
      tok0!)
%tok2 = linalg.matmul async [%tok0, %tok1] // waits for BOTH
```

6.3 Dependency Types (RAW, WAR, WAW)

Three classic data hazards, all encoded as token edges:

RAW (Read After Write): Compute reads buffer after DMA writes it. Most common. Must wait for write to complete.

WAR (Write After Read): DMA overwrites buffer with next tile while compute still reading current tile. Must wait for read to finish.

WAW (Write After Write): Two DMAs writing same buffer. Second waits for first to avoid corruption.

6.4 Why CPU Hazard Resolution Fails on NPUs

On CPUs, RAW/WAR/WAW are resolved *at runtime by hardware*:

Data forwarding / bypassing: Result from ALU output is wired directly to a dependent instruction's input, skipping register writeback. Requires a centralized pipeline with bypass muxes between stages.

Tomasulo / register renaming: WAR and WAW disappear by mapping architectural registers to a larger pool of physical registers. Each write targets a fresh physical register; concurrent reads of the old value are safe. Requires a centralized rename table and scoreboard.

Reorder buffer (ROB): Allows out-of-order execution with in-order commit. Requires centralized tracking of all in-flight operations.

None of these transfer to spatial architectures:

- **No central pipeline.** Data lives in distributed scratchpads across physically separate tiles. Cannot “forward” a 4KB buffer across a NoC the way a CPU forwards a 64-bit value through a bypass mux.
- **No centralized register file.** The “registers” are tile-local SRAM buffers managed by the compiler, not a hardware rename unit. No scoreboard watches all tiles simultaneously.
- **Wire delay at spatial scale.** A centralized ROB would need global visibility across dozens of independent tiles, each running its own DMA engine and compute engine. Physically impractical.

NPU resolution: compile-time token edges. The ACDG resolves all three hazards *statically*. The compiler determines ordering and inserts `air.token` SSA values that create happens-before constraints (paper p.25–26: dependencies “encoded directly in the IR using SSA token values”). Hardware simply obeys the token ordering—when a token is not yet produced, the dependent op waits. No speculation, no dynamic reordering.

	CPU	NPU (AIR)
When resolved	Runtime (hardware)	Compile time (compiler)
RAW	Data forwarding	Token edge: DMA→compute
WAR	Register renaming	Token edge: compute→next DMA
WAW	Register renaming	Token edge: DMA→DMA
Mechanism	Bypass mux, rename table, ROB	<code>air.token</code> SSA values
Why it works	Dynamic ILP extraction	Workload is regular, predictable

This is why the paper argues the compiler needs “dependency resolution close to hardware” (requirement 4, p.12)—the hardware will not solve it for you.

6.5 Three Kinds of Synchronization Edges

From Section 5.3.1 (p.12), the compute model defines three relationship types between operations controlled by synchronization lists:

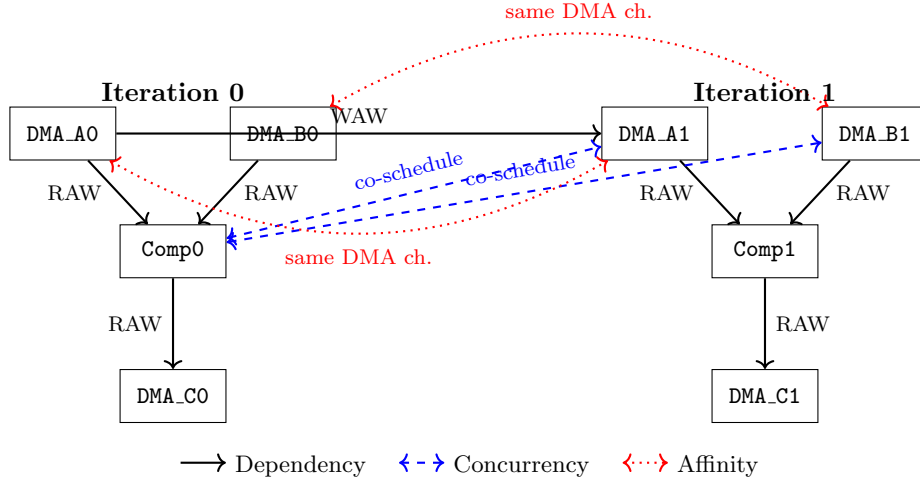
1. **Dependency lists** — *directed* edges. “A must finish before B starts” (happens-before). Encodes RAW/WAR/WAW hazards. Paper: “requires that all inputs to a operation that are modified by a source operation in the dependency list are visible before the sink operation is scheduled.”
2. **Concurrency lists** — *undirected* edges. “A and B must run at the same time” on exclusive resources. Paper: “each `air.token` in a concurrency list defines an undirected edge between two operations that indicates they must be scheduled at the same time. This implies that each operation must use exclusive resources.”
3. **Affinity lists** — *undirected* edges. “A and B must use the same physical resources” but at *different* times. Paper: “these operations’ time-slots must be disjoint, but the edge does not describe which operation must be scheduled first.”

6.5.1 Concrete Example: Double-Buffered Matmul Tile

Consider one tile computing $C += A \times B$ across two iterations with double buffering. Each iteration has four operations:

Op	Iteration 0	Iteration 1
Fetch A tile (L2→L1)	DMA_A0	DMA_A1
Fetch B tile (L2→L1)	DMA_B0	DMA_B1
Compute matmul	Comp0	Comp1
Write C tile (L1→L2)	DMA_C0	DMA_C1

The ACDG for this scenario contains all three edge types:



Reading the diagram:

- **Solid arrows (dependency):** DMA_A0→Comp0 is a RAW edge: compute can't start until the DMA finishes writing the A tile into L1. DMA_A0→DMA_A1 is a WAW edge: both write to the same L1 buffer (buf_A), so iteration 1's fetch must wait until iteration 0's fetch is consumed.
- **Dashed blue (concurrency):** Comp0↔DMA_A1 means these *must* run simultaneously—this is double buffering. While the compute engine processes tile 0's data, the DMA engine fetches tile 1's data into the *other* buffer. They use **exclusive** resources (compute engine vs. DMA engine), so no conflict. Without this edge, the scheduler could serialize them, killing the overlap.
- **Dotted red (affinity):** DMA_A0↔DMA_A1 means both must use the *same physical DMA channel*, but at different times. The compiler uses this to allocate DMA channels: these two ops share a channel, not compete for separate ones. Time-slots are disjoint (guaranteed by the WAW dependency edge), but the resource binding is the same.

6.5.2 Why Three Edges, Not Just Dependency?

Dependency alone can express *ordering* but not *resource binding* or *co-scheduling*. A compiler that only has dependency edges would:

- Miss the double-buffering overlap (concurrency): it would see DMA_A1 has no dependency on Comp0 and might schedule it *before* or *after*—but not constrain it to run *during*.
- Allocate DMA channels independently per operation (affinity): without knowing that DMA_A0 and DMA_A1 should reuse the same channel, it might allocate two channels or fail to reuse hardware.

6.6 Composing with Loops: Software Pipelining

Tokens as `scf.for` iteration arguments enable software pipelining:

```
scf.for %i = 0 to N iter_args(%prev_dma = %init_tok) {
  %tok0 = air.channel.get async [%prev_dma] → %buf
  %tok1 = matmul async [%tok0]
  %tok2 = air.channel.put async [%tok1]
  scf.yield %tok0    // next iter's DMA waits only for this DMA
}
```

Key: iteration $i+1$'s DMA waits only for iteration i 's DMA (via `%prev_dma`), *not* for i 's compute or store:

	DMA_get	compute	DMA_put
Iter 0:	[====]	[====]	[====]
Iter 1:	[====]	[====]	[====]
Iter 2:	[====]	[====]	

The pipeline overlap is expressed purely through which token gets passed as the loop-carried argument. Compiler sees the graph structure and can optimize pipeline depth and buffer allocation.

7 AIR Dialect Constructs in Use: Vector-Add Walkthrough

The paper (Section 6, p.13; Listing 7, p.33; Listing 3, p.14) demonstrates AIR on an element-wise vector add. This section starts from the plain algorithm, shows what the generic IR looks like, explains what's missing, then walks through the AIR output stage by stage.

7.1 Stage 0: The Algorithm (What We Actually Want to Compute)

```
// Element-wise vector add: out[i] = in1[i] + in2[i]
// 65536 float32 elements
for i in range(65536):
    out[i] = in1[i] + in2[i]
```

That's it. Three arrays, one loop, one add per element. $65536 \times 4 \text{ bytes} = 256 \text{ KB}$ per array, 768 KB total. No parallelism expressed, no tiling, no memory hierarchy—just the math.

The question: how does this get from a flat loop to running on a spatial NPU with distributed memories, DMA engines, and parallel tiles? Two stages:

1. **Generic MLIR** (Listing 7): upstream frameworks (Torch-MLIR, Triton) lower it to standard MLIR dialects with tiling and parallelism, but no spatial awareness.
2. **AIR IR** (Listing 3): MLIR-AIR transforms that into a fully spatial, async, explicitly-scheduled program.

7.2 Stage 1: Generic MLIR Input (Listing 7, p.33)

The upstream compiler has already done two things to the flat loop: **parallelized** it (split into 2 independent halves) and **tilled** it (process 1024 elements at a time into local buffers). But it uses only standard MLIR dialects (SCF, MemRef, Arith)—no spatial constructs. Paper p.13: “agnostic to the target hardware and does not yet reflect spatial execution, memory locality, or asynchronous scheduling.”

7.2.1 Step 1: Function Signature — Three Flat Buffers

```
func.func @eltwise_add(%arg0: memref<65536xf32>, // in1
                      %arg1: memref<65536xf32>, // in2
                      %arg2: memref<65536xf32>) { // out
```

Three flat 65536-element buffers. `memref<65536xf32>` = a pointer to contiguous memory. No hierarchy info—all three live in “some memory” with no distinction between host DRAM, L2, or L1 scratchpad.

What’s missing: on the NPU, `in1` starts in host DRAM (L3), tiles of it need to be in L2 (shared memory) and then L1 (tile-local scratchpad) before compute can touch it. This IR doesn’t express any of that.

7.2.2 Step 2: Parallelization — Split Work, But No Placement

```
%c2 = arith.constant 2 : index
scf.parallel (%arg3) = (%c0) to (%c2) step (%c1) {
```

`scf.parallel` with 2 iterations: each processes half the vector (32768 elements). This says “these iterations are independent and *may* run in parallel.”

What’s missing: `scf.parallel` is an *abstract* parallelism marker. It doesn’t say:

- *Which cores?* No tile coordinates, no grid shape.
- *How many?* A scheduler could run both iterations on one core sequentially—nothing prevents it.
- *Where physically?* No contiguity or affinity constraints.

7.2.3 Step 3: Tiling — Local Buffers, But Opaque Memory

```
%alloc    = memref.alloc() : memref<1024xf32, 2> // local buf for in1
  tile
%alloc_0   = memref.alloc() : memref<1024xf32, 2> // local buf for in2
  tile
%alloc_1   = memref.alloc() : memref<1024xf32, 2> // local buf for out
  tile
```

Three 1024-element local buffers. The 2 is a memory space annotation—in generic MLIR this is just an integer tag. It hints “not default address space” but carries **no semantic meaning**: the compiler doesn’t know if 2 means L1 scratchpad, L2 shared, or GPU shared memory.

What’s missing: on the NPU, L1 scratchpad on tile (0,0) is physically different from L1 on tile (0,1). A $1024 \times 4 = 4\text{KB}$ buffer must fit in the tile’s local SRAM. This IR can’t express that constraint.

7.2.4 Step 4: Data Movement — Coupled, Single-Sided

```
scf.for %arg4 = %c0 to %c65536 step %c2048 {
  %subview = memref.subview %arg0[%offset][1024][1]
              : memref<65536xf32> to memref<1024xf32>
  memref.copy %subview, %alloc
              : memref<1024xf32> to memref<1024xf32, 2>
```

`memref.subview` creates a window (pointer + offset) into the big array. `memref.copy` copies 1024 elements from that window into the local buffer.

What’s missing: `memref.copy` is **coupled**—one operation names *both* source and destination. The compiler can’t separately reason about:

- The “send” side (who pushes data onto the interconnect)
- The “receive” side (who pulls data into local memory)
- Which memory level each side lives at
- Whether the transfer is blocking or can overlap with compute

On the NPU, data movement from L3→L2 is a different DMA engine from L2→L1. `memref.copy` collapses both into one opaque operation.

7.2.5 Step 5: Compute — Correct, But Sequential

```
scf.for %arg5 = %c0 to %c1024 step %c1 {  
  %0 = memref.load %alloc[%arg5]      : memref<1024xf32, 2>  
  %1 = memref.load %alloc_0[%arg5]    : memref<1024xf32, 2>  
  %2 = arith.addf %0, %1 : f32  
  memref.store %2, %alloc_1[%arg5]    : memref<1024xf32, 2>  
}
```

Load from both input buffers, add, store to output buffer. 1024 iterations. Mathematically identical to the original $\text{out}[i] = \text{in1}[i] + \text{in2}[i]$, just operating on a tile.

What’s missing: everything here is **synchronous**. Each load blocks until complete. No overlap between this compute and any DMA. No tokens, no dependency tracking.

7.2.6 Step 6: Write-Back and Cleanup

```
%subview_3 = memref.subview %arg2[%offset][1024][1] ...  
memref.copy %alloc_1, %subview_3      // result → global  
memref.dealloc %alloc  
memref.dealloc %alloc_0  
memref.dealloc %alloc_1  
} // end scf.for (tiling loop)  
scf.reduce  
} // end scf.parallel
```

Copy result tile back to global output, deallocate local buffers. Again, `memref.copy` is coupled and blocking.

7.2.7 Summary of What Generic MLIR Captures vs. Misses

Property	Captured?	How
Algorithm correctness	✓	<code>arith.addf</code>
Tiling (1024-element tiles)	✓	<code>scf.for</code> + <code>memref.subview</code>
Parallelism (2 halves)	✓	<code>scf.parallel</code>
Spatial placement (which cores)	×	—
Memory hierarchy (L1/L2/L3)	×	2 is meaningless
Decoupled data movement	×	<code>memref.copy</code> is coupled
Async execution	×	everything blocking
Dependency tracking	×	implicit program order
Pipelining (DMA/compute overlap)	×	—

7.3 Stage 2: AIR-Transformed Output (Listing 3, p.14)

The AIR compiler transforms the generic MLIR into a program that makes spatial placement, memory hierarchy, data movement, async execution, and dependencies *all explicit*. Paper p.13: “makes the spatial and asynchronous execution explicit.” Same algorithm, same result—but the IR now tells the hardware *exactly* how to execute it.

We walk through the output top-to-bottom, explaining what each new construct adds and what it replaced from Stage 1.

7.3.1 Step 1: Channel Declarations (Replacing `memref.copy`)

```
air.channel @channel_0 [1, 2] // carries in1 tiles  
air.channel @channel_1 [1, 2] // carries in2 tiles  
air.channel @channel_2 [1, 2] // carries out tiles
```

Three named channels. Each has shape `[1, 2]`—one endpoint per worker in the herd (2 workers). These replace `memref.copy`’s coupled semantics with **decoupled put/get**: one actor pushes data onto a channel, another pulls from it, and they don’t need to name each other.

Channels are **symbolic** at this level—physical DMA paths, NoC routes, and buffer locations are determined during lowering to `mlir-ai`.

7.3.2 Step 2: Launch + Segment (Replacing `scf.parallel` Outer Scope)

```
%0 = air.launch async (%arg3, %arg4) in (1, 1) args(...) {
  %1 = air.segment @eltwise_add_0 async args(...) {
```

`scf.parallel` is gone. Replaced by two nested scopes:

- **air.launch**: host $\rightarrow$ accelerator boundary. Marks the L3 memory boundary. Everything inside runs on the NPU.
- **air.segment**: reserves a pool of compute resources with physical affinity. Marks the L2 memory boundary. All herds inside share L2.

What changed: `scf.parallel` said “independent iterations.” `air.launch + air.segment` say “offload to NPU, reserve these resources, L3 data lives outside, L2 data lives inside segment.”

Why only one segment? Vadd is trivially simple: 2 tiles doing the same operation, reading from the same input arrays, writing to the same output array. One L2 memory domain, one contiguous resource pool. No reason to split.

A segment reserves “pool(s) of compute resources...with physical affinity (e.g., resources in one chiplet)” (p.10). You need multiple segments when:

- **Multi-chiplet**: chiplet A handles rows 0–31, chiplet B handles rows 32–63. Each has its own L2 $\Rightarrow$ two segments.
- **Fused multi-kernel workloads**: e.g., a full attention block where one segment runs QK^T matmul on a 4×4 herd and another runs softmax on a 2×2 herd. Different resource needs, possibly different parts of the array.
- **Producer-consumer pipelines**: one segment produces intermediate results into L2, another segment consumes them. Each has its own resource lifetime—when segment A finishes, its tiles are released even while segment B is still running.

Workload	Segments	Why
Vadd (65K elements)	1	2 tiles, 1 L2, 1 operation
Attention block	2+	matmul + softmax need different herds
Multi-chiplet matmul	2+	each chiplet = separate L2 domain
Conv2d + BatchNorm fused	2	producer-consumer with different tile counts

7.3.3 Step 3: Segment-Level DMAs (L3 $\leftrightarrow$ L2)

```
%2 = air.channel.put async @channel_0[0,0]
      (%arg8[0,0,0][32,2,512][2048,1024,1])
%3 = air.channel.put async @channel_0[0,1]
      (%arg8[0,1024,0][32,2,512][2048,1024,1])
%4 = air.channel.put async @channel_1[0,0] (...)
%5 = air.channel.put async @channel_1[0,1] (...)
%6 = air.channel.get async @channel_2[0,0] (...)
%7 = air.channel.get async @channel_2[0,1] (...)
```

Six DMA operations at segment level. These replace the `memref.copy` calls from Stage 1, but with critical differences:

- **Decoupled:** `put` pushes data onto a channel from L3; matching `get` inside the herd pulls it into L1. Neither names the other.
- **Per-worker addressing:** `@channel_0[0,0]` feeds worker 0, `@channel_0[0,1]` feeds worker 1. Different offsets into `%arg8` (the global in1 buffer)—each worker gets its own slice.
- **Strided DMA descriptors:** `[32, 2, 512]` `[2048, 1024, 1]` are sizes and strides. The DMA hardware uses these to generate addresses automatically—no CPU-side loop needed.
- **All async, no inter-dependencies:** all six `put/get` ops have no token dependencies between them $\Rightarrow$ all can fire simultaneously on independent DMA channels.

7.3.4 Step 4: Herd Launch (Replacing `scf.parallel` Body)

```
%8 = air.herd @herd_0 async tile(%arg13, %arg14) in (1, 2) {
  %async_token, %results = memref.alloc() async
  %async_token_4, %results_5 = memref.alloc() async
  ... // 7 async allocs total
```

`scf.parallel` with 2 iterations becomes `air.herd` with a **1×2 grid**. Worker 0 is tile (0,0), worker 1 is (0,1).

What changed:

- `scf.parallel`: “2 independent iterations, run them somehow”
- `air.herd [1,2]`: “2 workers on a 1×2 grid of physically contiguous tiles, each with its own L1 scratchpad, atomically scheduled”

The 7 `memref.alloc() async` calls allocate L1 buffers on each tile. These replace the 3 `memref.alloc()` from Stage 1, but now: (a) there are more buffers (for double-buffering/pipelining), (b) they are `async` (allocation itself produces a token), and (c) they live in L1 implicitly because they are inside `air.herd` scope.

7.3.5 Step 5: Compute Loop with Pipelining (Replacing Sequential `scf.for`)

```
%9 = air.wait_all async [%async_token_10, %async_token_12]
%10:3 = scf.for %arg17 = 0 to 65536 step 4096
        iter_args(%arg18=%9, %arg19=..., %arg20=...) {
```

`air.wait_all` barriers on buffer allocations—compute can’t start until buffers exist (RAW on buffer memory).

The `scf.for` loop has **three loop-carried token arguments** (`%arg18, %arg19, %arg20`). These are three independent dependency chains enabling pipelining. In Stage 1, the loop had zero carried arguments—each iteration fully completed before the next started.

7.3.6 Step 6: Channel Gets Inside Herd (L2 $\rightarrow$ L1)

```
%11 = air.channel.get async [%arg18, %arg20]
      @channel_0[%arg13, %arg14] (%results_9[])
%12 = air.channel.get async [%arg18, %arg20]
      @channel_1[%arg13, %arg14] (%results_13[])
%13 = air.wait_all async [%11, %12]
```

Each worker pulls its input tiles from channels into L1 buffers. `@channel_0[%arg13, %arg14]` indexes by tile coordinates—worker (0,0) pulls from `@channel_0[0,0]`, matching the put at segment level.

`air.wait_all` after both gets: both input DMAs must finish before compute starts (RAW dependency). In Stage 1, this ordering was implicit because `memref.copy` was blocking.

7.3.7 Step 7: Async Compute (Same Math, Explicit Dependencies)

```
%14 = scf.for %arg21 = 0 to 1024 step 1
      iter_args(%arg22 = %13) {
        %t20, %r21 = memref.load async [%arg22] %results_9[%arg21]
        %t22, %r23 = memref.load async [%arg22] %results_13[%arg21]
        %22 = arith.addf %r21, %r23 : f32
        %t24 = memref.store async [%arg22] %22, %results_7[%arg21]
        %23 = air.wait_all async [%t20, %t22, %t24]
        scf.yield %23
      }
```

Same `arith.addf` as Stage 1. Same 1024-iteration inner loop. But every load and store is now `async` with explicit token dependencies. The `air.wait_all` at the end collects all tokens before yielding to the next iteration.

7.3.8 Step 8: Write-Back and Final Sync

```
%15 = air.channel.put async [%14]
      @channel_2[%arg13,%arg14] (%results_7[])
...
scf.yield %15, %20, %21 // three tokens to next iteration
```

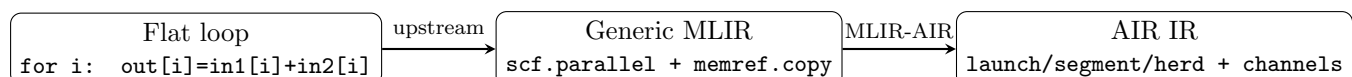
After compute finishes (%14 token), result is pushed L1→L2 via `channel_2`. Three tokens yielded for next iteration’s pipelining: %15 (output DMA done), %20 and %21 (buffer reuse deps).

```
} // end herd
air.wait_all [%2, %3, %4, %5, %6, %7, %8]
return
} // end segment, launch
```

`air.wait_all` with all 7 tokens (6 segment-level DMAs + herd)—launch doesn’t complete until everything finishes. In Stage 1, this was implicit because `scf.parallel + scf.reduce` blocked until all iterations finished.

7.4 Stage 3: What the Compiler Added

Aspect	Stage 1 (Generic MLIR)	Stage 2 (AIR)
Placement	<code>scf.parallel</code> —no cores	<code>air.herd [1,2]</code> —2 physical tiles
Scope hierarchy	Flat	<code>launch</code> $\supset$ <code>segment</code> $\supset$ <code>herd</code>
Memory hierarchy	<code>memref<..., 2></code> —opaque tag	L1/L2/L3 by nesting scope
Data movement	<code>memref.copy</code> —coupled, blocking	<code>channel.put/get</code> —decoupled, async
Dependencies	Implicit (program order)	Explicit <code>air.token</code> on every op
Pipelining	None—sequential iterations	3 loop-carried tokens
Parallelism	Abstract “independent iterations”	6 concurrent DMAs + 2 tiles



The programmer writes Stage 0. Upstream frameworks produce Stage 1. MLIR-AIR produces Stage 2. The programmer never touches spatial constructs—the compiler adds all placement, data movement, hierarchy, and dependency information automatically.

8 Scheduling Constructs

8.1 The Problem: Where Does Computation Run?

On a CPU, you write code and the OS schedules it onto cores—you never think about placement. On a GPU, you write a kernel and the hardware scheduler maps thread blocks to SMs automatically. On a spatial NPU, neither works:

- There is no OS scheduler—tiles are bare-metal, each with its own instruction memory.
- There is no hardware thread-block scheduler—the compiler must decide *which tiles* run *which code* at *which time*.
- Resources (tiles, L2 memory, DMA engines) must be explicitly reserved and released—no automatic sharing.

AIR solves this with a three-level hierarchy that mirrors the physical hardware: **launch** (host $\leftrightarrow$ accelerator) $\supset$ **segment** (resource pool with L2) $\supset$ **herd** (compute grid with L1). Think of it as: **job** $\rightarrow$ **department** $\rightarrow$ **team**.

8.2 `air.launch` — Host-to-Accelerator Boundary

Plain English: “send this work to the NPU.” Like CUDA’s `kernel<<<>>> launch`, but for spatial hardware. Everything inside runs on the accelerator; everything outside stays on the host CPU.

Key properties:

- **Optional iteration space:** multiple independent instances. “Completely parallel schedule is legal” and instances “must not rely on observing the effects of any other instance” (p.9).
- **Multiple compiled variants:** compiler emits different versions for different resource availability. “Our lowering for AMD NPU architecture uses this freedom to offer different opportunistic granularities (e.g., one column or whole array)” (p.10).
- **Resource lifetime management:** once nested ops finish, launch releases hardware resources back to runtime.

8.3 `air.segment` — Resource Reservation

Plain English: “reserve this chunk of the NPU for my work.” A segment claims a pool of tiles + their shared L2 memory + the DMA engines connecting them. While a segment holds these resources, no other segment can use them. When the segment finishes, resources are released.

Think of it as booking a conference room: you get the room (L2 memory), the chairs inside (tiles), and the whiteboard (DMA engines). Others can’t walk in until you leave.

Reserves a **pool of compute resources** with physical affinity. Groups cores that share L2 memory and local interconnects.

Why separate from herd? “An architecture might want to define two pools of resources that have physical affinity (e.g., resources in one chiplet) so that they can ensure that the scheduler dispatches `air.herd` operations within that segment exclusively using the segment resources” (p.10).

Optional grid space for replication: `air.segment [2]` = two identical resource pools dispatched concurrently (e.g., left and right half of NPU).

8.4 `air.herd` — Parallel Workers on Physical Cores

Plain English: “run this code on a grid of tiles simultaneously.” If `air.launch` is “send work to NPU” and `air.segment` is “reserve resources,” then `air.herd` is “now actually do the computation on these specific tiles.” Every worker in the herd runs the same code body but is specialized by its tile coordinates—just like CUDA threads use `threadIdx` to pick their slice of work.

Group of workers executing concurrently on a grid of compute tiles. Closest analog: CUDA thread blocks, but with physical placement guarantees that CUDA doesn’t give.

Key properties:

- **Index space:** worker coordinates (`%x`, `%y`) like CUDA `threadIdx`. Same code, specialized by index.
- **Atomic scheduling:** “only scheduled when resources for *all* the workers are available” (p.10). No partial herds—avoids deadlocks.
- **Physically contiguous:** “workers are allocated as a physically local contiguous block” (p.10). 4×2 herd = 4×2 rectangle of adjacent tiles. Enables cascade (neighbor-to-neighbor) communication.
- **Multiple herds default sequential:** use `air.token` to make concurrent if desired.

8.5 Hierarchy Maps to Memory Hierarchy

AIR construct	Scope	Memory level
<code>air.launch</code>	Host $\leftrightarrow$ accelerator	L3 / DRAM
<code>air.segment</code>	Resource pool	L2 (memory tiles)
<code>air.herd</code>	Compute grid	L1 (core scratchpad)

9 Data Locality Constructs

9.1 The Problem: No Caches, No Automatic Data Movement

On a CPU, when your code reads `array[i]`, the cache hierarchy transparently fetches the data from DRAM into L3, then L2, then L1—you never think about it. On a GPU, shared memory (`__shared__`) requires manual management, but global memory accesses are still cached.

On a spatial NPU, **there are no caches at all**. Each tile has a small scratchpad (L1, typically 32–64 KB). Shared memory tiles provide L2. DRAM sits behind shim tiles at the edge of the array. If your compute tile needs data, someone must explicitly:

1. Program a DMA engine to copy data from DRAM into L2
2. Program another DMA to copy from L2 into the tile’s L1 scratchpad
3. After compute, reverse the process for results

The compiler must express all of this explicitly in the IR. AIR provides three levels of abstraction for this: `MemRef` annotations (where data lives), `air.memcpy` (transitional coupled movement), and `air.channel` (the real decoupled primitive).

9.2 Memory Hierarchy via MemRef Annotations

Plain English: tag each buffer with *which memory* it lives in. Standard MLIR `memref` defaults to global memory. AIR adds memory space annotations that actually mean something:

```
%global = memref<1024xf32>           // default: DRAM (L3)
%l2_buf = memref<256xf32, memory_space=L2> // shared memory tile
%l1_buf = memref<64xf32, memory_space=L1>  // core-local scratchpad
```

Three levels mapped to NPU hardware:

- **L3 / DRAM** — off-chip, behind shim tiles, huge, slow
- **L2 / cluster scratchpad** — memory tiles, shared by column of cores
- **L1 / private** — on compute tile, per-core, small, fastest

9.3 `air.memcpy` — Transitional Abstraction

Plain English: “copy this data between memory levels, and here’s which levels.” It’s an enhanced `memref.copy` that carries metadata about hierarchy levels and data layout. Think of it as an intermediate step—the compiler uses it while lowering from generic MLIR, before splitting it into decoupled put/get. You rarely see it in final AIR output.

Key capability: **in-flight data reshaping**. “Decoupling the logical layout of a MemRef from its physical representation in memory” (p.11). DMA hardware supports stride-based reshaping—e.g., copy from L3 to L2 while rearranging row-major to tiled layout.

`air.memcpy` is intermediate; lowered to `air.channel` operations:

```
air.memcpy (%src, %dst)           // high level
// lowered to:
air.channel.put @chan (%src)      // decouple into put + get
air.channel.get @chan (%dst)
```

9.4 `air.channel` — Core Data Movement Primitive

Plain English: the real way data moves on the NPU. Instead of one `memcpy` that names both source and destination (coupled), you split it: one actor **puts** data onto a named channel, another actor **gets** from that channel. They don’t name each other—just the channel. This is like a conveyor belt: the warehouse puts boxes on, the factory floor takes boxes off. Neither knows or cares about the other’s internals.

Stream-based transfer between distinct memory regions. Decoupled into a **put** (producer) and **get** (consumer) linked through a named symbolic channel.

From p.11:

- **put**: transfers data from a source address onto a serialized stream
- **get**: retrieves data from the stream into a destination address at a particular memory level

Why decouple into put/get instead of one memcpy?

1. **Producer and consumer live at different hierarchy levels.** The **put** sits in the segment body (L2 scope), the **get** inside the herd body (L1 scope). Each op is local to its memory scope:

```

air.segment {
    %l2_buf = memref.alloc() : memref<256xf32, L2>
    air.channel.put @chan (%l2_buf)          // L2 scope

    air.herd [%x, %y] in [4, 1] {
        %l1_buf = memref.alloc() : memref<64xf32, L1>
        air.channel.get @chan (%l1_buf) // L1 scope
        // compute on l1_buf
    }
}

```

2. **Cross-level references.** A put at launch level (L3) can feed a get inside a herd (L1). The channel abstracts however many hops the data takes.
3. **Independent optimization.** Compiler can schedule put early (prefetch), delay get, optimize each side’s DMA path independently.

Broadcasting: One put feeds multiple gets. Specified through affine integer sets mapping source to destination indices. Lowers to NoC multicast on AMD NPU.

Channel bundles: Group related channels—e.g., @chan_A[0], @chan_A[1] for different tiles of matrix A.

MemRef integration: air.channel ops use same offset/size/stride semantics as standard MLIR, so existing analysis passes still work.

9.5 How Scheduling and Data Locality Interact

Where data ops appear in the hierarchy tells the compiler which memory level they operate on:

```

air.launch {
    air.channel.put @chan_in (%dram_buf)          // L3 boundary
                                                    // L3 → stream

    air.segment {
        %l2 = memref.alloc() : memref<..., L2>    // L2 boundary
        air.channel.get @chan_in (%l2)             // stream → L2
        air.channel.put @chan_local (%l2)          // L2 → stream

        air.herd [%x, %y] in [4,1] {
            %l1 = memref.alloc() : memref<..., L1> // L1 boundary
            air.channel.get @chan_local (%l1)       // stream → L1
            // compute on %l1
        }
    }
}

```

Data cascades $L3 \rightarrow L2 \rightarrow L1$. Each transfer is a separate put/get pair at the right scope. Compiler sees the full picture: can optimize each level independently, pipeline transfers across levels, detect broadcast opportunities.

Additionally, **air.herd** guarantees physical contiguity—neighboring workers can write to each other’s local memory via cascade connections without going through L2 (p.12). And **air.segment** ensures “accesses to shared memories remain localized within their data producers and consumers, reducing unnecessary data movement” (p.12).

10 Synchronization Constructs (Complete)

Two constructs handle synchronization in AIR: **air.token** (explicit dependency edges) and **air.channel** (implicit back-pressure sync). They work at different levels and complement each

other.

10.1 `air.token` — Explicit Dependency Control

When any operation is marked `async`, it returns an `air.token` SSA value signaling its completion. This token can then be consumed by other operations to express “don’t start until this finishes” (Section 5.3.1, p.12).

Granularity is tunable. Paper p.12: “fine-grained control over execution dependencies in asynchronous workloads, with the granularity tunable from coarse-grained synchronization over regions of code to fine-grained per-operation scheduling.” You can put one token on an entire `air.launch` (coarse) or one per individual DMA op (fine).

10.1.1 Mechanism 1: `air.wait_all` — Blocking Barrier

Paper p.12: “explicit `air.wait_all` operations, can be inserted into synchronous control flow. This allows us to prevent further operation dispatch until tokens specified in the `air.wait_all` operation have signaled completion.”

```
%tok0 = air.channel.get async @chanA, %bufA
%tok1 = air.channel.get async @chanB, %bufB
air.wait_all [%tok0, %tok1]           // BLOCKS until both DMAs done
linalg.matmul %bufA, %bufB, %bufC    // only runs after wait passes
```

This is a hard barrier—nothing after it executes until all listed tokens fire. Simple but conservative.

10.1.2 Mechanism 2: Synchronization Lists — Scheduling Graph

Paper p.12: “synchronization lists, that explicitly specify the relative scheduling of operations in time and space, describe a scheduling graph. Backend lowerings can use this graph to push dependency resolution to distributed dispatchers in hardware devices.” Instead of one blocking wall, you build a graph of “who waits for whom” that the hardware scheduler interprets directly.

10.2 Three Types of Synchronization Edges

The compute model defines three types of relationships between operations (p.12):

10.2.1 Dependency Lists — Directed, “Happens-Before”

Paper p.12: “Dependency lists encode directed edges between an operation and the predecessor operations in which its inputs are defined. It requires that all inputs to an operation that are modified by a source operation in the dependency list are visible before the sink operation is scheduled.”

```
%tok0 = air.channel.get async ...           // DMA writes bufA
%tok1 = linalg.matmul async [%tok0] ...    // dep list: [%tok0]
// tok1 can't start until tok0 finishes (RAW dependency)
```

This encodes RAW/WAR/WAW hazards as directed edges in the ACDG.

10.2.2 Concurrency Lists — Undirected, “Must Co-Schedule”

Paper p.12: “Each `air.token` in a concurrency list defines an undirected edge between two operations that indicates they must be scheduled at the same time. This implies that each operation must use exclusive resources.”

This is for *guaranteeing* overlap, not just allowing it. Example: double buffering where DMA fills buffer B while compute processes buffer A. You want to *require* they run simultaneously on separate resources (separate DMA engine and compute unit).

10.2.3 Affinity Lists — Undirected, “Same Resources, Different Time”

Paper p.12: “These are lists of tokens that define undirected edges between operations that must execute using the same resources. In practice, this means these operations’ time-slots must be disjoint, but the edge does not describe which operation must be scheduled first.”

Opposite of concurrency: these ops share a resource (e.g., same DMA engine), so they can’t overlap, but either could go first. The affinity edge says “same resource, you figure out the order.”

10.3 Tokens in Loops

10.3.1 scf.for — Single Token (Fig 4a)

One token carried across iterations. Fully sequential—each iteration waits for the previous one’s entire body:

```
%tf = scf.for %i = 0 to %N iter_args(%ta) {
    %t1 = air.op async [%ta]
    %t2 = air.op async [%t1]
    scf.yield %t2    // pass to next iteration
}
```

10.3.2 scf.for — Multiple Tokens (Fig 4b)

Paper p.17: “An arbitrary number of `air.token` values can be carried across iterations, each representing an independently progressing thread of execution.”

```
%tf = scf.for %i iter_args(%ta, %tb, %tc, %td) {
    %t1 = air.op async [%ta, %tc]
    %t2 = air.op async [%t1, %td]
    %t3 = air.op async [%ta, %t1]
    %t4 = air.op async [%t1, %t2]
    scf.yield %t1, %t2, %t3, %t4    // four independent streams
}
```

Four independent dependency chains flowing across iterations. Some chains from iteration $i+1$ can start while others from iteration i still run. Paper p.17: “This enables precise modeling of parallel pipelines, race conditions, and shared resource usage.”

10.3.3 scf.parallel — Fork/Join (Fig 4c)

Paper p.17: “MLIR-AIR supports asynchronous dependencies within structured parallel execution constructs such as `scf.parallel`.”

```
%init = air.wait_all async
%tp = scf.parallel (%i) = (0) to (%N) init (%t1) {
    %td = air.memcpy async [%t1] (%y[%i], %a[%i])
    %tc = air.op async [%td]
    scf.reduce(%tc) {
        ^bb0(%lhs, %rhs):
        %tj = air.wait_all async [%lhs, %rhs]
        scf.reduce.return %tj
    }
}
```

```
}
```

Key points:

- **Synchronized start:** `air.token` in `init` argument starts all parallel threads together
- **Synchronized end:** `air.wait_all` reduction tree collects all tokens—all iterations must finish before the region completes
- **Within each iteration:** async dependencies work normally

10.4 `air.channel` as Synchronization

Channels are not just data movement—they also synchronize (Section 5.3.2, p.13).

Paper p.13: “an `air.channel.get` operation is synchronized to an `air.channel.put` with the same `air.channel` by back pressure.”

Back pressure means: the channel is a bounded buffer. If the producer pushes data and the consumer hasn’t consumed previous data, the producer **stalls**. No explicit token needed—the channel itself enforces ordering:

```
// In segment body:
air.channel.put @chan (%l2_data)    // pushes data

// In herd body:
air.channel.get @chan (%l1_buf)     // implicitly waits for put
// No token connects these -- @chan IS the sync mechanism
```

Paper p.13: “This synchronization abstraction—combined with asynchronous dependencies on `air.channel` actors to enforce synchronization local to each memory space—is both simple and effective: enforcement of dependencies between distributed data movers does not require complex control-flow dependencies across code regions.”

Without channel back-pressure, you’d need explicit tokens crossing from segment scope into herd scope. Channel back-pressure handles it implicitly.

10.5 Grouped Completion via `air.launch`

Paper p.13: “an `air.launch`’s `air.token` is released only when *all* operations within the `air.launch` have completed.”

`air.launch` produces a single token that fires when everything inside (segments, herds, DMAs) finishes. The host synchronizes on “entire accelerator job done” without tracking individual ops.

10.6 Hardware Pipelining: Ping-Pong Buffering

Section 7.4.1 (p.20) describes the concrete payoff. The loop is unrolled by 2, yielding “ping” and “pong” threads with four loop-carried tokens:

- 2 tokens for producer-consumer dataflow (ping stage and pong stage)
- 2 tokens for intra-stage resource contention (producer-side and consumer-side)

The flattened ACDG (Fig 7, p.20) shows the interleaved pipeline:

Time →	
Ping:	[produce] → [consume] → [produce] → [consume]
Pong:	[produce] → [consume] → [produce] →

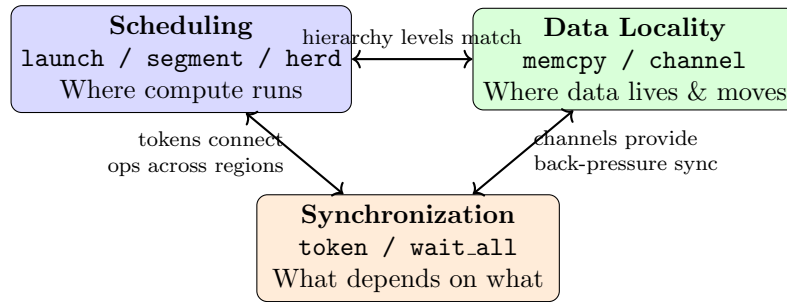
While ping consumer processes data, pong producer fills next buffer. Paper p.20 validates: “data reads and writes execute concurrently across two buffers, validating the correctness and effectiveness of the pipelined ACDG transformation.”

10.7 Synchronization Summary

Mechanism	Type	Scope	How
Token + dep list	Explicit, directed	Any ops	A before B
Token + concurrency	Explicit, undirected	Co-scheduled	Must overlap
Token + affinity	Explicit, undirected	Resource-sharing	Same resource, no overlap
<code>air.wait_all</code>	Explicit barrier	Sync flow	Block until all fire
Channel back-pressure	Implicit	Cross-hierarchy	Get waits for put
Launch completion	Implicit group	Host boundary	Fires when all inside done

Tokens are the **fine-grained** mechanism (per-operation). Channels are the **cross-hierarchy** mechanism (across segment/herd boundaries). Together they express any pattern from fully sequential to deeply pipelined, and backend lowerings push it all to hardware dispatchers.

11 Summary: The AIR Mental Model



These three categories of constructs together let AIR represent:

- **Concurrency control** — which cores run what, when
- **Memory placement** — which data lives where in the hierarchy
- **Data movement** — explicit DMA ops, visible and schedulable
- **Synchronization** — explicit dependency edges, not implicit ordering

All as first-class, compiler-visible operations in the MLIR framework.